



I. PORTADA

Tema:	Ejercicio PostgreSQL
Unidad de Organización Curricular:	PROFESIONAL
Nivel y Paralelo:	5° - "A"
Alumno:	Mazabanda Pilamunga Diego Abraham
Asignatura:	Sistemas de Bases de Datos Distribuidos
Docente:	Ing. Caiza Rubén Mg.

II. INFORME DEL TALLER

III. OBJETIVOS

General:

Desarrollar y aplicar consultas avanzadas en PostgreSQL mediante la creación de tablas relacionadas, inserción de datos, uso de subconsultas, fragmentación de datos por departamentos y construcción de vistas, con el fin de fortalecer el manejo de operaciones SQL que optimicen la organización, análisis y visualización de la información en una base de datos relacional.

Específico:

- Diseñar y crear las tablas necesarias para representar las entidades departamentos y empleados, aplicando claves primarias y foráneas para mantener la integridad referencial de la base de datos.
- Implementar subconsultas que permitan obtener información relevante, como empleados con salarios superiores al promedio, departamentos sin personal y promedios salariales por área.
- Aplicar la fragmentación horizontal y la creación de vistas, con el propósito de organizar los datos por departamentos y consolidar la información en una vista unificada que facilite las consultas globales.

IV. INSTRUCCIONES

- Crear las tablas
- Insertar datos de prueba
- Realizar subconsultas
- Comentar cada consulta
- Validar resultados
- Documentar el proyecto

V. EJERCICIOS SUBCONSULTAS

```
CREATE TABLE departamentos (  
    id INT PRIMARY KEY,  
    nombre VARCHAR(50)  
);
```

```
CREATE TABLE empleados (  
    id INT PRIMARY KEY,  
    nombre VARCHAR(50),  
    salario DECIMAL(10,2),
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE TECNOLOGÍAS DE LA INFORMACIÓN
CICLO ACADÉMICO: AGOSTO – ENERO 2026



departamento_id INT,
FOREIGN KEY (departamento_id) REFERENCES departamentos(id)
);
Insertar datos
--- Departamentos
INSERT INTO departamentos (id, nombre) VALUES
(1, 'Recursos Humanos'),
(2, 'TI'),
(3, 'Marketing');

	id [PK] integer	nombre character varying (50)
1	1	Recursos Humanos
2	2	TI
3	3	Marketing

Ilustración 1. Tabla Departamentos

---Empleados
INSERT INTO empleados (id, nombre, salario, departamento_id) VALUES
(1, 'Ana Pérez', 2500.00, 1),
(2, 'Luis Gómez', 3000.00, 2),
(3, 'Marta Díaz', 2800.00, 1),
(4, 'Carlos Ruiz', 3200.00, 2),
(5, 'Sofía Torres', 2700.00, 3),
(6, 'Jorge León', 2900.00, 3);

	id [PK] integer	nombre character varying (50)	salario numeric (10,2)	departamento_id integer
1	1	Ana Pérez	2500.00	1
2	2	Luis Gómez	3000.00	2
3	3	Marta Díaz	2800.00	1
4	4	Carlos Ruiz	3200.00	2
5	5	Sofía Torres	2700.00	3
6	6	Jorge León	2900.00	3

Ilustración 2. Tabla Empleados

SUBCONSULTAS

- Empleado con salario mayor al promedio:
Obtén todos los empleados cuyo salario es mayor al salario promedio de todos los empleados.
SELECT *



```
FROM empleados
WHERE salario > (SELECT AVG(salario)
FROM empleados);
```

```
----Empleado con salario mayor al promedio:
SELECT *
FROM empleados
WHERE salario > (SELECT AVG(salario)
FROM empleados);
```

Ilustración 3. Subconsulta cuyo salario mayor al promedio

	id [PK] integer	nombre character varying (50)	salario numeric (10,2)	departamento_id integer
1	2	Luis Gómez	3000.00	2
2	4	Carlos Ruiz	3200.00	2
3	6	Jorge León	2900.00	3

Ilustración 4. Empleados con mayor salario al promedio

Explicación: Calcula el promedio de sueldos y luego lista a los empleados que ganan más de ese valor.

- Departamentos sin empleados:

Lista los nombres de los departamentos que no tienen empleados asignados.

```
SELECT *
FROM departamentos
WHERE id NOT IN(SELECT departamento_id
FROM empleados);
```

```
--Departamentos sin empleados:
SELECT *
FROM departamentos
WHERE id NOT IN(SELECT id_departamento
FROM empleados);
```

Ilustración 5. Subconsulta sin empleados



id	nombre
[PK] integer	character varying (50)

Ilustración 6. Departamento sin empleados

Explicación: Busca en la tabla de departamentos cuáles no aparecen en la lista de empleados.

- Empleado con salario más alto:

Muestra el nombre y salario del empleado que gana más.

SELECT nombre , salario

FROM empleados

WHERE salario = (SELECT MAX(salario)

FROM empleados);

```
-----Empleado con salario más alto:
SELECT nombre , salario
FROM empleados
WHERE salario = (SELECT MAX(salario)
                  FROM empleados);
```

Ilustración 7. Subconsulta empleados con salario más alto

	nombre	salario
	character varying (50)	numeric (10,2)
1	Carlos Ruiz	3200.00

Ilustración 8. Empleado con salario más alto

Explicación: Encuentra el valor máximo de sueldo y muestra al empleado que lo tiene.

- Salario promedio por departamento:

Para cada empleado, muestra: nombre, salario y el salario promedio de su departamento.

SELECT nombre, salario, (SELECT AVG(salario)

FROM empleados

WHERE departamento_id = e.departamento_id) AS promedio_departamento

FROM empleados e;



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE TECNOLOGÍAS DE LA INFORMACIÓN
CICLO ACADÉMICO: AGOSTO – ENERO 2026



```
--Salario promedio por departamento:  
SELECT nombre, salario, (SELECT AVG(salario)  
FROM empleados  
WHERE departamento_id = e.departamento_id) AS promedio_departamento  
FROM empleados e;
```

Ilustración 9. Consulta salario promedio por departamento

	nombre character varying (50) 🔒	salario numeric (10,2) 🔒	promedio_departamento numeric 🔒
1	Ana Pérez	2500.00	2650.0000000000000000
2	Luis Gómez	3000.00	3100.0000000000000000
3	Marta Díaz	2800.00	2650.0000000000000000
4	Carlos Ruiz	3200.00	3100.0000000000000000
5	Sofía Torres	2700.00	2800.0000000000000000
6	Jorge León	2900.00	2800.0000000000000000

Ilustración 10. Empleados con salario promedio por departamento

Explicación: A cada empleado le calcula el promedio de sueldos de su departamento y lo muestra al lado.

- Departamentos con promedio mayor al promedio general:
Lista los departamentos cuyo promedio de salario es mayor al promedio general de la empresa.

```
SELECT d.nombre,AVG(e.salario)  
FROM departamentos d  
JOIN empleados e ON d.id = e.departamento_id  
GROUP BY d.nombre  
HAVING AVG(e.salario)>(SELECT AVG(salario)  
FROM empleados);
```

```
---Departamentos con promedio mayor al promedio general  
SELECT d.nombre,AVG(e.salario)  
FROM departamentos d  
JOIN empleados e ON d.id = e.departamento_id  
GROUP BY d.nombre  
HAVING AVG(e.salario)>(SELECT AVG(salario)  
FROM empleados);
```

Ilustración 11. Consulta departamento



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE TECNOLOGÍAS DE LA INFORMACIÓN
CICLO ACADÉMICO: AGOSTO – ENERO 2026



	nombre character varying (50)	avg numeric
1	TI	3100.00000000000000000000

Ilustración 12. Departamento con promedio mayor al promedio general

Explicación: Compara el promedio de cada departamento con el promedio total y muestra solo los que están por encima.

Fragmentamos la tabla empleados según el departamento

Una tabla solo para Recursos Humanos (RRHH)

Una tabla solo para TI

Una tabla solo para Marketing

--Recursos Humanos (RRHH)

CREATE TABLE empleados_RRHH

(id INT PRIMARY KEY,

nombre VARCHAR(50),

salario DECIMAL(10,2),

departamento_id INT);

```
CREATE TABLE empleados_RRHH
(id INT PRIMARY KEY,
nombre VARCHAR(50),
salario DECIMAL(10,2),
departamento_id INT);
```

Ilustración 13. Tabla empleados_RRHH

--TI

CREATE TABLE empleados_TI

(id INT PRIMARY KEY,

nombre VARCHAR(50),

salario DECIMAL(10,2),

departamento_id INT);

```
CREATE TABLE empleados_TI
(id INT PRIMARY KEY,
nombre VARCHAR(50),
salario DECIMAL(10,2),
departamento_id INT);
```

Ilustración 14. Tabla empleados_TI

--Marketing

CREATE TABLE empleados_marketing



(id INT PRIMARY KEY,
nombre VARCHAR(50),
salario DECIMAL(10,2),
departamento_id INT);

```
CREATE TABLE empleados_marketing  
(id INT PRIMARY KEY,  
nombre VARCHAR(50),  
salario DECIMAL(10,2),  
departamento_id INT);
```

Ilustración 15. Tabla empleados_marketing

```
-- Insertamos según condición  
INSERT INTO empleados_RRHH  
SELECT * FROM empleados WHERE id_departamento = 1;  
Visualizamos los datos  
SELECT * FROM empleados_RRHH;
```

	id [PK] integer	nombre character varying (50)	salario numeric (10,2)	departamento_id integer
1	1	Ana Pérez	2500.00	1
2	3	Marta Díaz	2800.00	1

Ilustración 16. Verificación de la inserción de datos en la tabla empleados_RRHH

```
INSERT INTO empleados_TI  
SELECT * FROM empleados WHERE id_departamento = 2;  
Visualizamos los datos  
SELECT * FROM empleados_TI;
```

	id [PK] integer	nombre character varying (50)	salario numeric (10,2)	departamento_id integer
1	2	Luis Gómez	3000.00	2
2	4	Carlos Ruiz	3200.00	2

Ilustración 17. Verificación de la inserción de datos en la tabla empleados_TI

```
INSERT INTO empleados_marketing  
SELECT * FROM empleados WHERE id_departamento = 3;  
Visualizamos los datos  
SELECT * FROM empleados_marketing;
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE TECNOLOGÍAS DE LA INFORMACIÓN
CICLO ACADÉMICO: AGOSTO – ENERO 2026



	id [PK] integer	nombre character varying (50)	salario numeric (10,2)	departamento_id integer
1	5	Sofía Torres	2700.00	3
2	6	Jorge León	2900.00	3

Ilustración 18. Verificación de la inserción de datos en la tabla empleados_marketing

Creación de vista

```
CREATE VIEW Todos_Empleados AS
```

```
(SELECT * FROM empleados_RRHH
```

```
UNION ALL
```

```
SELECT * FROM empleados_TI
```

```
UNION ALL
```

```
SELECT * FROM empleados_marketing);
```

```
CREATE VIEW Todos_Empleados AS  
(SELECT * FROM empleados_RRHH  
UNION ALL  
SELECT * FROM empleados_TI  
UNION ALL  
SELECT * FROM empleados_marketing);
```

Ilustración 19. Creación de vistas

Visualizamos la vista

```
SELECT * FROM Todos_Empleados;
```

	id integer	nombre character varying (50)	salario numeric (10,2)	departamento_id integer
1	1	Ana Pérez	2500.00	1
2	3	Marta Díaz	2800.00	1
3	2	Luis Gómez	3000.00	2
4	4	Carlos Ruiz	3200.00	2
5	5	Sofía Torres	2700.00	3
6	6	Jorge León	2900.00	3

Ilustración 20. Visualización de la vista



VI. CONCLUSION

Es importante destacar que la práctica permitió comprender de manera integral el funcionamiento de las subconsultas y la fragmentación de datos en PostgreSQL, demostrando cómo estas herramientas facilitan el análisis y la organización de la información dentro de una base de datos relacional. Además, la creación de vistas permitió integrar los datos fragmentados en una sola estructura lógica, optimizando el acceso y la visualización de la información. Finalmente, se reforzó el uso de sentencias SQL para realizar consultas eficientes, mantener la integridad de los datos y aplicar principios de diseño que mejoran la administración y el rendimiento del sistema de bases de datos.